

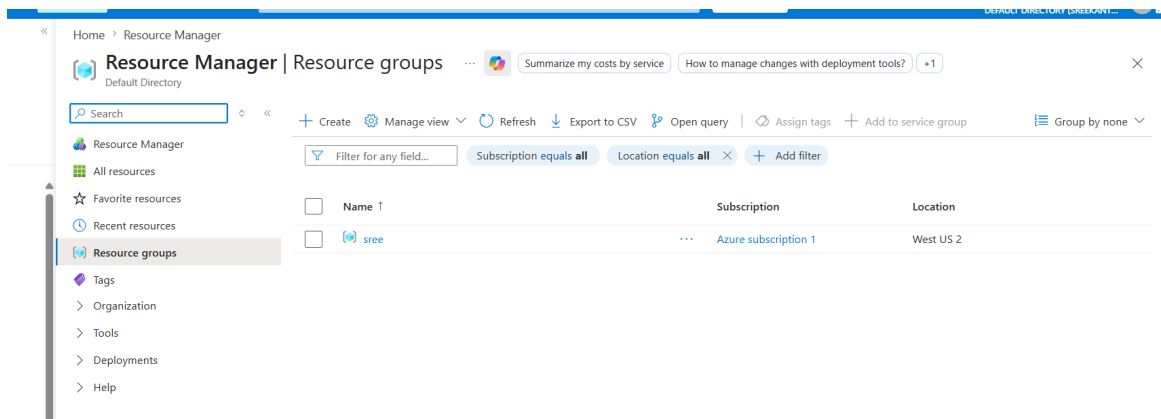
On-Premises to Azure SQL Database Migration

using Azure Data Factory (ADF)

Recently worked on migrating an on-premises MySQL database to Azure SQL Database using Azure Data Factory (ADF). Below is the end-to-end, step-by-step approach I followed, along with screenshots from the actual implementation.

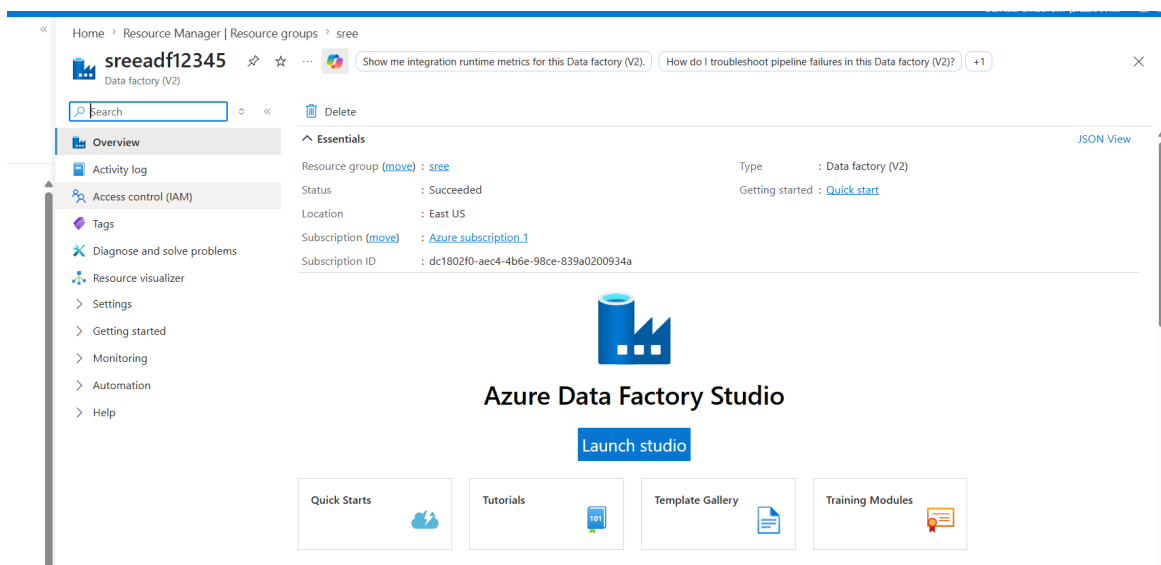
Step 1: Created a Resource Group

A Resource Group in Azure acts as a logical container that holds all related resources for a project. Creating this first keeps every migration-related resource — Data Factory, SQL Database, Integration Runtime, etc. — organized under one roof, making management, access control, billing, and cleanup much easier.



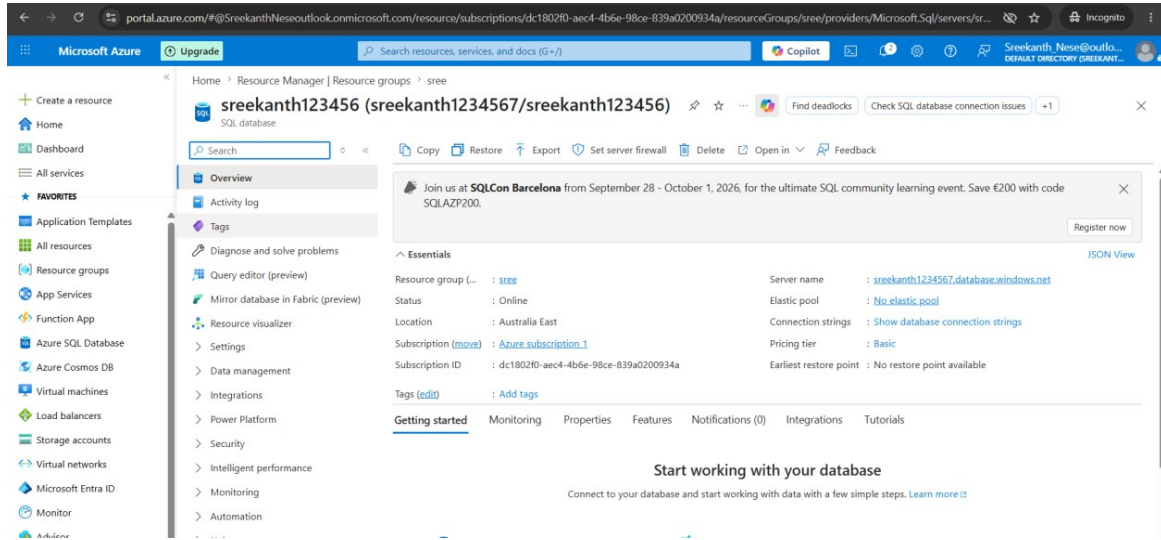
Step 2: Created the Azure Data Factory Service

Azure Data Factory (ADF) is a cloud-based ETL and data integration service. It was provisioned to orchestrate the entire migration — from reading data out of the on-prem source system to loading it into Azure SQL Database. The Data Factory Studio is where all pipelines, datasets, and linked services are built and managed.



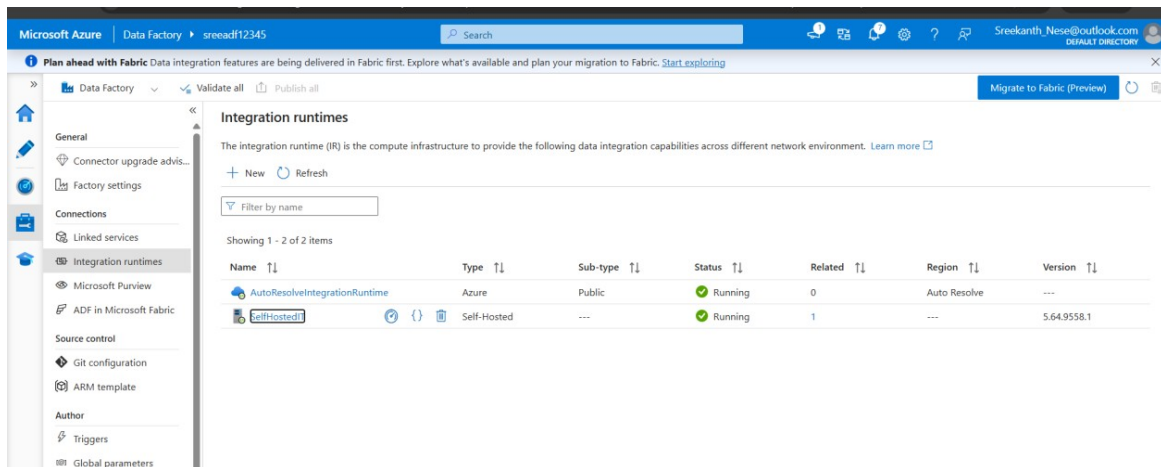
Step 3: Created the Azure SQL Database

This is the destination for the migrated data. Azure SQL Database is a fully managed relational database service in the cloud that hosts all the tables being migrated from on-premises, with built-in high availability, backups, and scaling.



Step 4: Created a Self-Hosted Integration Runtime (SHIR)

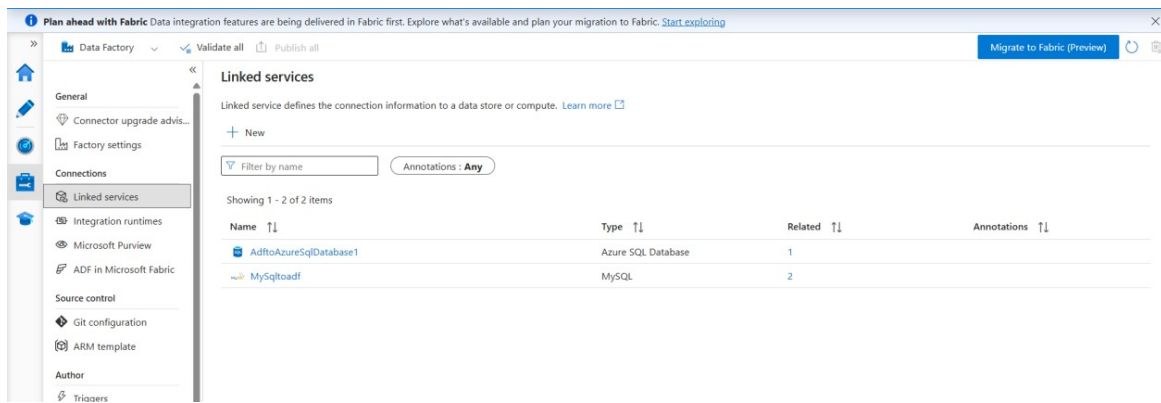
Since the source database is on-premises and not publicly accessible over the internet, a Self-Hosted Integration Runtime was installed on a local machine inside the on-prem network. It acts as a secure bridge, allowing Azure Data Factory (running in the cloud) to reach into the on-prem network and connect to the source database safely — without exposing it directly to the internet.



Step 5: Created Linked Services

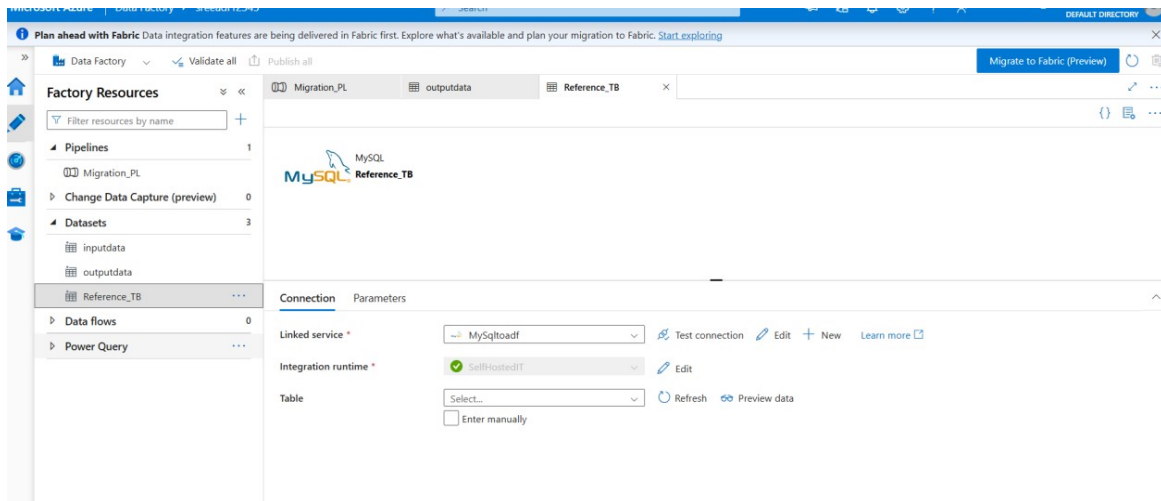
Linked Services define the connection information ADF needs to connect to each data store. Two linked services were created:

- One pointing to the on-premises MySQL database, connected through the Self-Hosted Integration Runtime
- One pointing to the Azure SQL Database (the migration destination)



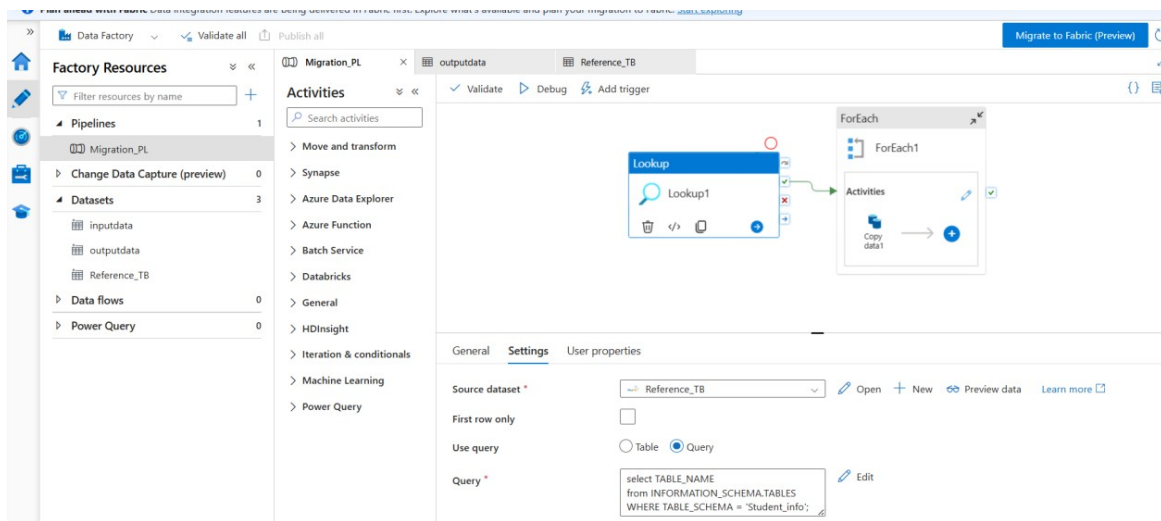
Step 6: Created a Reference / Control Table

Before building the pipeline, a control table was created in the on-prem MySQL database containing metadata — specifically, the list of table names — for every table that needed to be migrated. This control table is what makes the pipeline dynamic: instead of hardcoding table names inside the pipeline, the pipeline reads this table at runtime and figures out what to migrate.



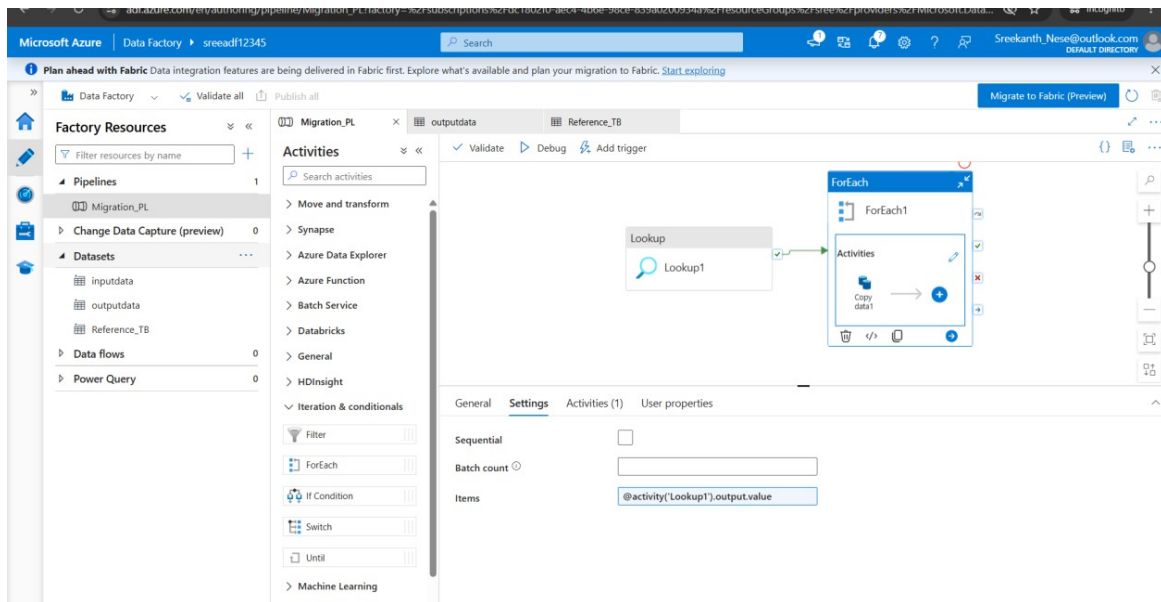
Step 7: Built the Pipeline — Lookup Activity

A new pipeline was created in ADF. The first activity added was a Lookup activity, configured to run a query against the control table (using INFORMATION_SCHEMA.TABLES filtered by schema) and return the full list of table names that need to be copied.



Step 8: Added a ForEach Activity

A ForEach activity was added right after the Lookup activity. It takes the list of table names returned by the Lookup (`@activity('Lookup1').output.value`) and loops through each one, triggering the inner activities once per table. This is what allows a single pipeline to handle any number of tables without duplicating logic.

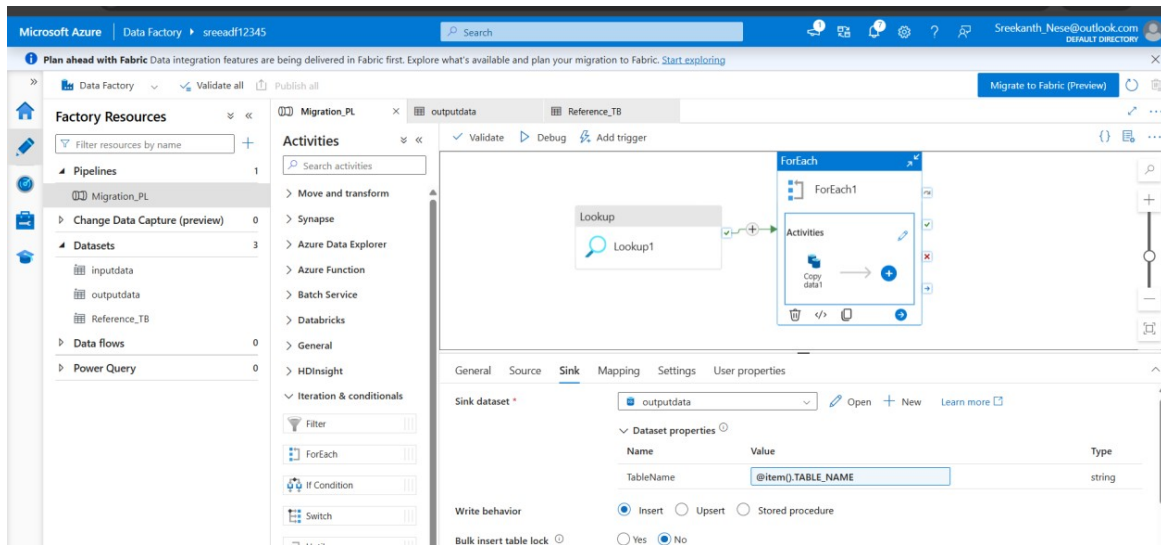
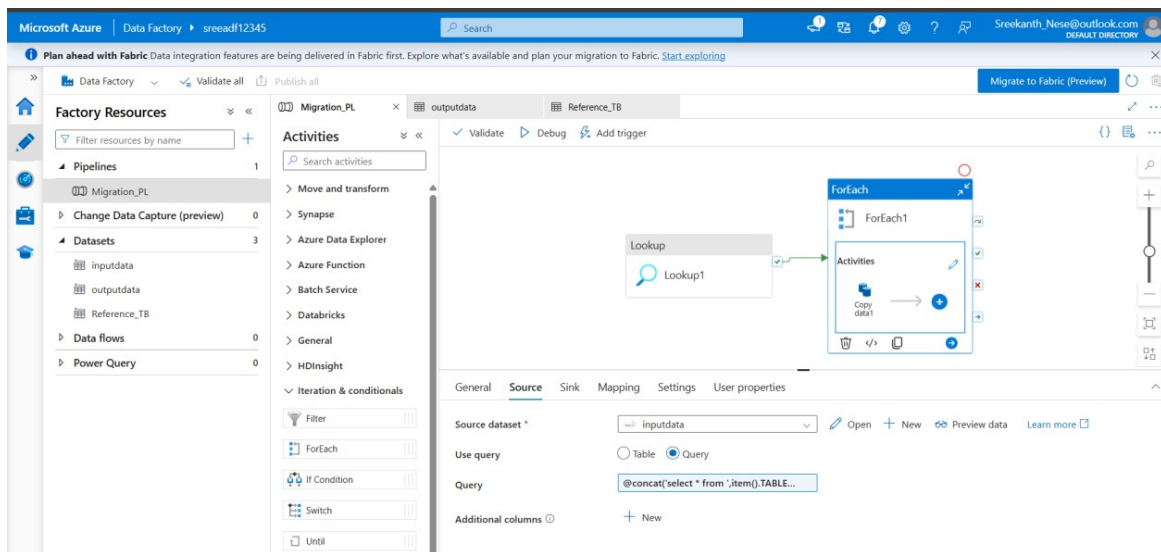


Step 9: Configured the Copy Activity Inside the Loop

Inside the ForEach loop, a Copy Data activity was added and configured with dynamic source and sink datasets:

- Source: a dynamic query built from the current item, e.g. `@concat('select * from ', item().TABLE_NAME)`
- Sink: the Azure SQL Database table, with the table name set dynamically via `@item().TABLE_NAME` and write behavior set to Insert

This single Copy activity ran once for every table returned by the Lookup — handling anywhere from 100 to 200+ tables in one pipeline execution, without needing a separate Copy activity per table.



Step 10: Triggered the Pipeline

With the pipeline fully built and validated, it was run on demand using the "Trigger Now" option in ADF. This kicks off the Lookup → ForEach → Copy sequence immediately.

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime	User properties	Activity run ID
Lookup1	Succeeded	Lookup	7/1/2026, 10:26:55 PM	16s	SelfHostedIT		5996d178-c0a5
ForEach1	Succeeded	ForEach	7/1/2026, 10:27:12 PM	1m 3s	SelfHostedIT		93176bb7-75c1
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	55s	SelfHostedIT		ef776c54-d2ac
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	26s	SelfHostedIT		d9080770-1b19
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	35s	SelfHostedIT		5927f036-36e1
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	44s	SelfHostedIT		f365995e-55c2
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	52s	SelfHostedIT		d9881bc1-0561
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	27s	SelfHostedIT		aedf0d1e-a257
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	30s	SelfHostedIT		e4b3aa1e-4cb4
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	35s	SelfHostedIT		3d27b53d-ecae
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	43s	SelfHostedIT		6577f0f9-b27c
Copy data1	Succeeded	Copy data	7/1/2026, 10:27:12 PM	1m 0s	SelfHostedIT		1ec9d91c-23b6

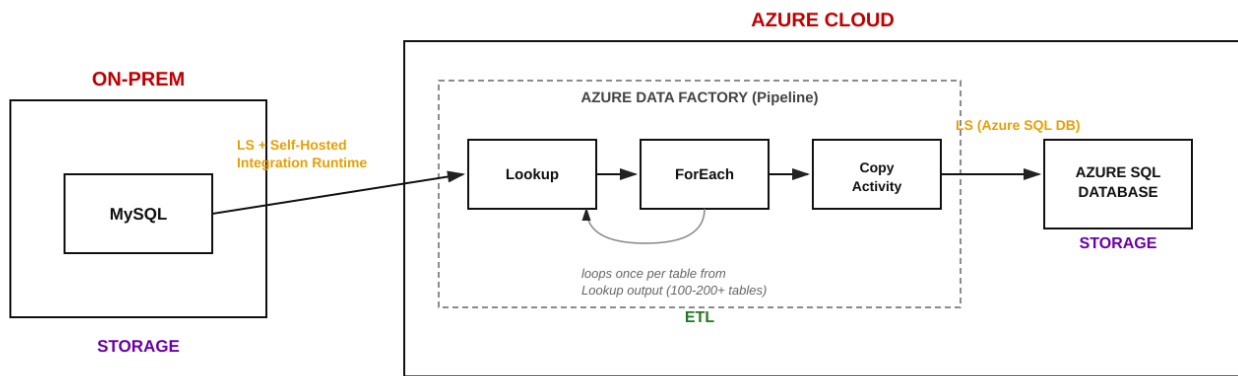
Step 11: Validated the Execution

The pipeline run was monitored in the ADF monitoring tab, where every Lookup, ForEach, and Copy activity showed a Succeeded status along with its duration and the Self-Hosted Integration Runtime used. Finally, the migrated tables were verified directly in Azure SQL Database using the built-in Query Editor, confirming that all data had moved over successfully.

attendance_id	enrollment_id	attendance_date	status	mode
1	46	2025-03-18	Late	Offline
2	190	2024-02-05	Late	Online
3	8	2024-09-06	Present	Online
4	115	2024-03-08	Absent	Online
5	124	2024-11-26	Present	Online
6	64	2025-07-31	Late	Online
7	72	2024-08-23	Present	Offline

Architecture Diagram

The diagram below summarizes the overall flow: the on-prem MySQL database is reached via the Self-Hosted Integration Runtime, the ADF pipeline (Lookup → ForEach → Copy Activity) drives the migration, and data lands directly in Azure SQL Database — with the Copy activity looping once per table for every table returned by the Lookup.



Key Takeaway

Using a metadata-driven design (Lookup + ForEach + Copy activity) made the pipeline scalable and reusable. Instead of hardcoding table names, the pipeline dynamically reads them from a control table — meaning new tables can be added to future migrations simply by adding a row to that table, with zero changes to the pipeline itself.